

Глава 1. Сборка датасета, приведение данных к одному формату, предобработка и EDA

1. Я нашла несколько готовых датасетов по hh.ru и собрала их в табличку **Датасеты_hh**. Все найденные датасеты почему-то из 2023 года, кроме первого набора, который насчитывает данные с 2005.
2. Все таблички из первого набора данных (с сайта IEEEE) были объединены в одну, получилось csv на 15 ГБ, который постоянно виснул. Поэтому было решено отказаться от первого набора данных, но объединить остальные в один большой датасет.
3. В качестве отправной точки был выбран датасет [Raw Jobs Data from Head Hunters Russia](#) (далее – RawJobs) поскольку в нем представлено больше всего вакансий. Соответственно, при объединении с ним других датасетов, я в большей степени ориентировалась на то, какие колонки есть в RawJobs. Однако, поскольку колонки (1) salary, (2) experience, (3) job_type содержали в себе комбинированные данные о нескольких показателях (типа от 160000 до 160000 RUR), их содержимое было разбито на следующие колонки соответственно: (1) salary_from (int), salary_to (int), currency (str); (2) experience_from (int), experience_to (int), no_experience (Boolean); (3) job_type_concrete (str), schedule (str). Оригинальные столбцы salary, experience и job_type были удалены, остальные столбцы остались без изменений.
4. Если это было необходимо, каждый другой датасет обрабатывался таким образом, чтобы в нем появлялись столбцы с аналогичным содержанием. Любые другие столбцы, которых не было в датасете RawJobs при конкатенации датасетов удалялись.
5. Столбец *location* был предобработан так, чтобы осталось только информация о городе (был создан новый столбец *city*).
6. Столбец *date_of_post* был предобработан так, чтобы осталось только информация о месяце, когда был сделан пост.

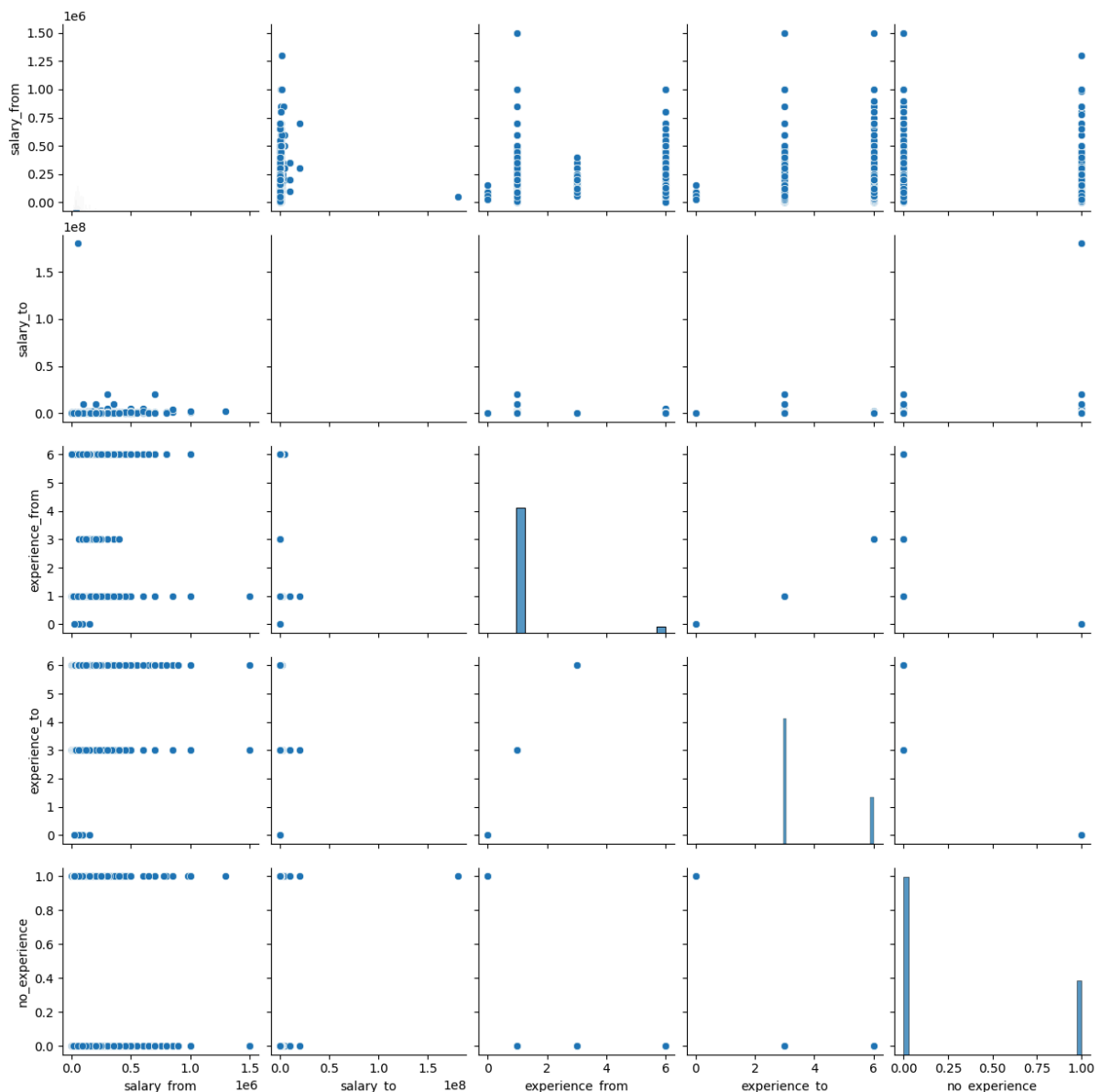
Предобработка

1. В датасете было много пропущенных значений:

Процент пропущенных значений по столбцам:

```
№: 11.04%
id: 0.00%
title: 0.00%
description: 0.00%
key_skills: 37.78%
company: 0.00%
location: 34.68%
date_of_post: 0.00%
type: 10.89%
salary_from: 39.07%
salary_to: 69.78%
currency: 43.30%
experience_from: 47.45%
experience_to: 27.47%
no_experience: 0.00%
job_type_concrete: 0.00%
schedule: 0.16%
city: 0.00%
description_clean: 0.00%
title_key: 0.00%
```

2. Поскольку переменная `salary_from` является целевой, я попробовала заполнить пропуски в этом столбце, сделав следующее:
 - a. подсчитать среднее по столбцу `salary_from` (далее переменная **`mean_sal`**)
 - b. посчитать среднее `salary_from` по каждой компании (`company`) и каждой должности
 - i. для классификации по должностям столбец `title` был преобразован так, что из него извлекалось первое существительное (а также слова типа 'управляющий /управляющая'), чтобы свести должности типа 'главный механик завода' и 'ответственный механик' к одной – 'механик'. Новая должность заносилась в столбец `title_key`.
 - c. рассчитать коэффициент, на который предлагаемая зарплата по этой должности (далее переменная **`title_coef`**) или в этой компании (далее переменная **`company_coef`**) в среднем отличается от средней предлагаемой `salary_from`
 - d. записать коэффициенты `salary_from_company_coefficient` и `salary_from_title_coefficient` по каждой строке в отдельные столбцы. В строках, где расчет этих коэффициентов был невозможен, оставить NaN
 - e. в строках с пропусками в столбце `salary_from` пропуски заполнить произведением **`mean_sal*title_coef*company_coef`**
3. К сожалению, заполнение пропусков этим алгоритмом заняло больше недели (в течение которой было заполнено менее половины пропусков).
4. Поэтому было принято решение все строки с пропусками в столбце `salary_from` удалить. Также были удалены все строки с нероссийской валютой, а также строки с работой в нероссийских городах, где з/п являлась явным выбросом на фоне российских зарплат.
5. Все строковые значения во всех столбцах были приведены к нижнему регистру.
6. Пропуски в колонке 'city' были заполнены самым частотным значением по данной компании.
7. Категориальные переменные (`city`, `company`, `title_key`) были перекодированы так, чтобы самые частотные значения остались, а остальные -- слились в категорию `other`. При этом сохраняем такие категории, у которых наиболее экстремальные значения `salary_from` -- экстремально высокие и экстремально низкие.
8. Была рассмотрена взаимосвязь между числовыми переменными:



Из приведенного выше графика можно сделать вывод, что нижний порог зарплаты прямо скоррелирован с верхним порогом зарплаты.

У тех, у кого опыт работы насчитывает более 6 лет, бывает более высокая зарплата, чем у тех, у кого опыт работы составляет более одного года. Однако, наблюдений о более промежуточных вариантах нет. Кроме того, в вакансиях, требующих опыта работы от 6 лет, в основном предлагается такая же зарплата, как в вакансиях, требующих опыта от 1 года.

С максимальным допустимым опытом работы аналогичная связь: в вакансиях, требующих до 6 лет опыта работы, встречаются обещания более высокой зарпалвты, чем в вакансиях, предполагающих не более года опыта работы.

Ожидаемым образом, в вакансиях, не требующих опыта работы, обещания о размере заработной платы, как правило, меньше, чем в вакансиях с требованиями об опыте работы.

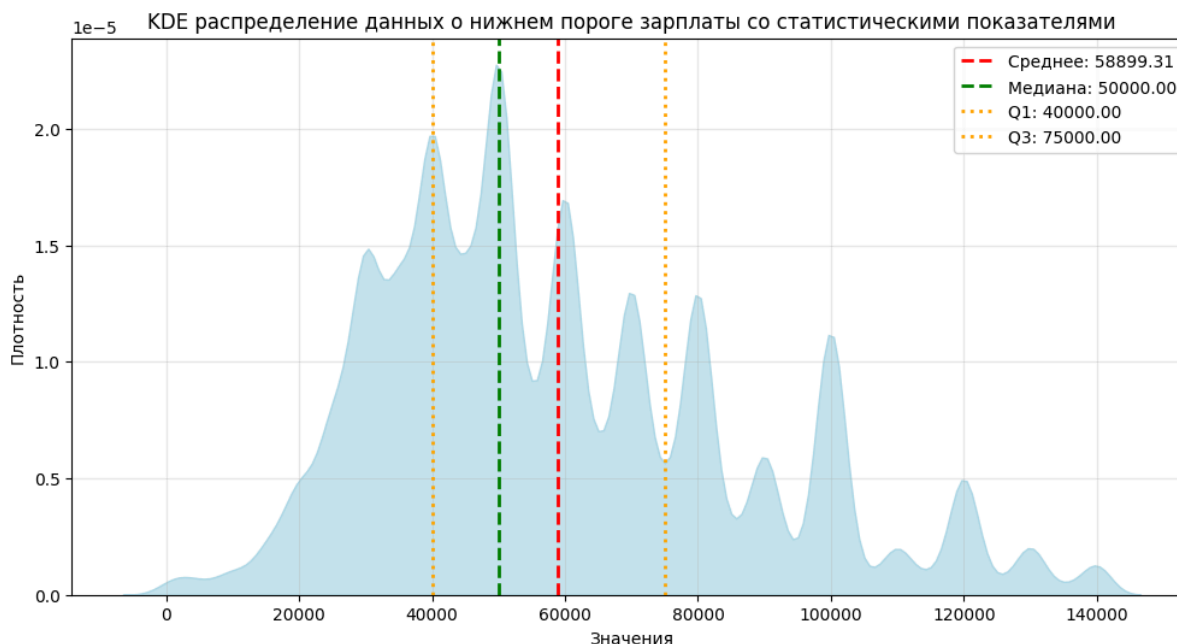
9. Была рассмотрена связь между категориальными переменными и целевой:

Согласно приведенным графикам, самая высокая средняя зарплата предлагается в Сочи, Южно-Сахалинске и Москве. То, что Сочи и Южно-Сахалинск обогнали Москву может быть результатом того, что в Сочи и Южно-Сахалинске предлагаются в основном вакансии определенного типа. Например, в Сочи в основном предлагаются вакансии агента по недвижимости с довольно высокой оплатой. В то время как в Москве предлагаются разнообразные вакансии.

Примечательно, что первые места по среднему размеру з/п среди работодателей занимают не известные компании (которые тоже присутствуют в рейтинге довольно высоко) — СБЕР, РЖД, Яндекс — а менее известные предприятия. Самую большую зарплату предлагают в вакансиях, в названии которых главное слово — брокер, директор, геодезист, руководитель и т.д.

Вакансии с самой высокой зарплатой почему-то предлагали в августе и сентябре. Кроме того, самая большая зарплата — у вакансий с вахтовым методом работы (2-е место — удаленная работа) и у проектной работы. Станным образом, значительную сумму предлагают волонтерам. Возможно, эти данные нужно изучить лучше или совсем удалить.

10. Был рассмотрен непосредственно распределение целевой переменной.



11. Далее все категориальные признаки были закодированы с помощью One-hot-Encoding'a, а неинформативные колонки ('Unnamed', '№', 'id', 'title', 'description', 'key_skills', 'location', 'date_of_post', 'type', 'salary_to', 'currency', 'description_clean', 'salary_from_company_coefficient', 'salary_to_company_coefficient', 'salary_from_title_coefficient', 'salary_to_title_coefficient', 'salary_from_filled') были удалены.

12. Все наблюдения в полученном датафрейме были нормализованы.

Глава 2. Простые модели

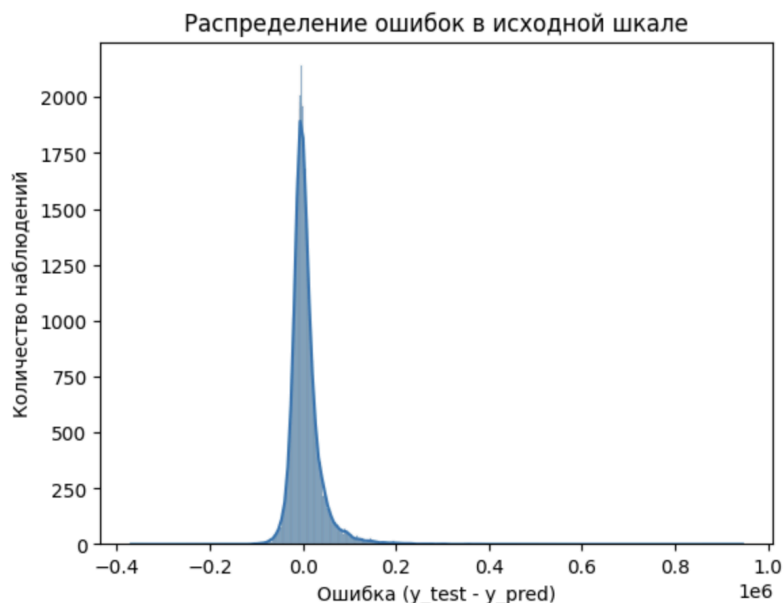
Предварительно все признаки проверены на наличие пропусков. Пропуски в колонках `experience_from` (47% пропусков) и `experience_to` (32% пропусков) заполнены самым частотным значением (1 и 3 соответственно).

Таргетная переменная была логарифмически преобразована через натуральный логарифм ($1 + y$)

Перед обучением данные нормализовались через `MinMaxScaler`

Линейная регрессия без обработки колонки *description* — то есть только не-текстовые признаки

	Train RMSE	Test RMSE	Train R^2	Test R^2	
<code>experience_from, experience_to, no_experience, city (OHE), company (OHE), title_key (OHE)</code>					
<code>LinearRegression</code>	37652.97	38178.77	0.3537	0.3427	
<code>Ridge</code>	37848.52	38260.12	0.3470	0.3399	
<code>Lasso</code>	48158.62	48421.52	-0.0572	-0.0572	



Линейная регрессия без обработки колонки *description* с обрезкой выбросов в целевой переменной по правилу межквартильного размаха

	Train RMSE	Test RMSE	Train R^2	Test R^2	
--	------------	-----------	-------------	------------	--

experience_from, experience_to, no_experience, city (OHE), company (OHE), title_key (OHE)					
LinearRegression					
Ridge	28659.07	28585.12	0.4085	0.4085	
Lasso	38471.12	38424.39	-0.0658	-0.0688	

Линейная регрессия без обработки колонки *description* с PCA

Попробовала применить PCA и оставить только 115 признаков вместо 236

	Train RMSE	Test RMSE	Train R ²	Test R ²	
experience_from, experience_to, no_experience, city (OHE), company (OHE), title_key (OHE)					
LinearRegression	66178.40	66253.65	0.3126	0.3159	
Ridge	66174.81	66250.01	0.3126	0.3159	
Lasso	56509.71	56509.71	-0.0572	-0.0572	

Случайный лес

	Train RMSE	Test RMSE	Train R ²	Test R ²	
experience_from, experience_to, no_experience, city (OHE), company (OHE), title_key (OHE)					
n_estimators=100, max_depth=None, min_samples_split=2, min_samples_leaf=1, max_features=1.0	17925.16	38901.79	0.8535	0.3176	
n_estimators=100, max_depth=1,min_samples_split=5, min_samples_leaf=5, max_features=0.5	37214.84	38344.02	0.3687	0.3370	

n_estimators=120, max_depth=200, min_samples_split=2, min_samples_leaf=1, max_features=1.0,	20750.15	38275.01	0.8037	0.3394	
---	----------	----------	--------	--------	--

Градиентный бустинг

	Train RMSE	Test RMSE	Train R ²	Test R ²	
experience_from, experience_to, no_experience, city (OHE), company (OHE), title_key (OHE)					
n_estimators =100, max_depth=1, learning_rate=0.1, subsample=0.8, random_state =42	26684.48	36929.38			

Линейная регрессия с description BOW

Добавим к данным для обучения данные колонки description, закодированные через BOW (max_features=1000, stop_words=russian_stop_words из NLTK)

experience_from, experience_to, no_experience, city (OHE), company (OHE), title_key (OHE) + description (BOW)					
LinearRegression	33671.05	9842533092890.77	0.4832	-43681993904886520.0000	
Ridge	33750.12	34288.49	0.4807	0.4699	

Линейная регрессия с description TIF-IDF

Добавим к данным для обучения данные колонки description, закодированные через TIF-IDF (max_features=1000, min_df=2, max_df=0.8, stop_words=russian_stop_words из NLTK)

experience_from, experience_to, no_experience, city (OHE), company (OHE), title_key (OHE) + description (TF-IDF)					
LinearRegression	33670.72	19579438662293.48	0.4832	-172857870973219008.0000	
Ridge	33750.12	34288.49	0.4807	0.4699	

Глава 3. Простые NNs

Все расчеты выполняются на лемматизованном столбце description, который закодирован с помощью TF-IDF (max_features=1000,min_df=2,max_df=0.8), то есть на датасете со следующей конфигурацией:

experience_from, experience_to, no_experience, city (OHE), company (OHE), title_key (OHE) + description (TF-IDF)

- все пропуски в датасете заполнены модой
- выбросы удалены больше 99 перцентиля
- данные нормализованы (как в X, так и в y)
- y нормализован
- criterion = torch.nn.MSELoss()
- optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
- 10 эпох
- все обучается на 0.5 данных, иначе ОЗУ переполнена и все вылетает

Ниже указывается R2 и RMSE на валидационной выборке на 10-ой эпохе в зависимости от конфигурации

	без столбца description	со столбцом description	только description
Один слой	(234-1) R ² : 0.3418, RMSE: 33770.64	(1233-1) R ² : 0.4524, RMSE: 30674.09	(1000-1) R ² : 0.3732, RMSE: 32887.04
Два слоя через ReLu	(234-117-1) R ² : 0.3717, RMSE: 32995.29	(1233 -- 617 -- 1) R ² : 0.5660, RMSE: 27306.08	(1000-500-1) R ² : 0.4885, RMSE: 29708.91
Два слоя через ReLu и DropOut	(234-117-1) R ² : 0.3674, RMSE: 33108.10	(1233 -- 617 -- 1) R ² : 0.5969, RMSE: 26316.54	(1000-500-1) R ² : 0.5210, RMSE: 28748.96
Четыре слоя через ReLu и DropOut	(234-128-64-1) R ² : 0.3559, RMSE: 33405.91	(1233 - 1024 -- 512 -- 256) R ² : 0.4893, RMSE: 29621.95	(1000-512-256-64) R ² : 0.5393, RMSE: 28194.52
Четыре слоя через ReLu и DropOut и BatchNorm	(234-128-64-1) R ² : 0.3609, RMSE: 33276.29	(1233 - 1024 -- 512 -- 256) R ² : 0.5600, RMSE: 27496.25	

Глава 4. Предобученные эмбединги

Только эмбединги

cointegrated/rubert-tiny

Один слой (312-1)	R ² : 0.2598, RMSE: 0.8233
Четыре слоя (312 - 256 -- 128 -- 64)) через ReLu и DropOut и BatchNorm	R ² : 0.3208, RMSE: 0.7887